

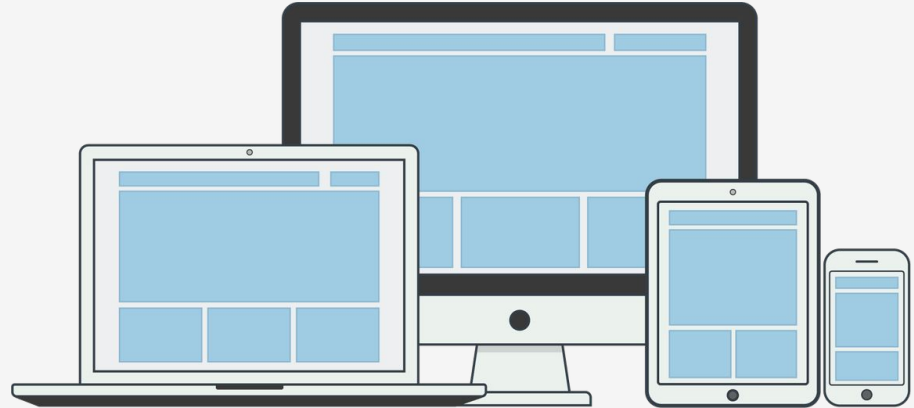
High Level Synthesis for FPGA: Bridging Software and Hardware

Ikramul Hassan Sazid

Hamm-Lippstadt University of Applied Sciences

Hardware / Software Codesign - SS26

Prof. Dr. Achim Rettberg



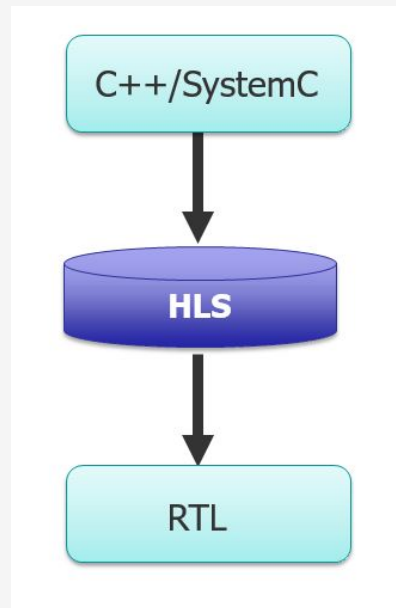
INTRODUCTION: High-Level Synthesis (HLS)

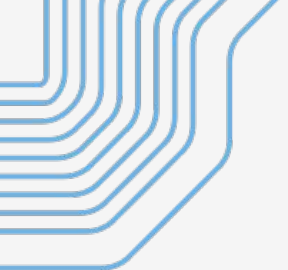
The Driving Motivation: The complexity of System-on-Chip (SoC). High level coding to significantly improve developer productivity.

Core Concept Defined: HLS is the automated process that translates untimed or partially timed high-level specifications (like C, C++, or SystemC) into a low-level, cycle-accurate RTL representation.

Output Definition: The final output of the HLS compilation process is a synthesizable RTL netlist.

Target Implementation: On FPGA similar to RTL netlist

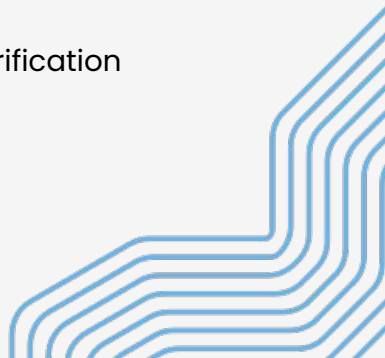


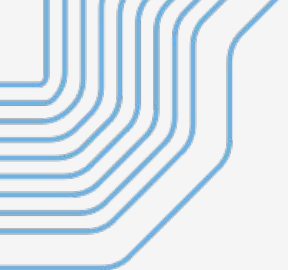


History and Previous Limitations of HLS

Early Origins (1980s–1990s): Early HLS initiatives (often called behavioral compilers) attempted to raise the abstraction level. However, first-generation tools primarily used hardware description languages (HDLs) as inputs rather than software languages.

Key Limitations of Early Tools:

- *Poor Quality of Results (QoR):* highly variable and unpredictable RTL, inefficient resource allocation.
 - *Misaligned Target Audience:* RTL designers (resisted paradigm shift) instead of empowering software and algorithm engineers.
 - *Validation Challenges:* Unreliable timing behaviors in the generated RTL, design verification extremely difficult.
- 



The FPGA Perspective

- Generates RTL specifically optimized to map onto a fixed, discrete fabric ie. LUTs, Flip-Flops (FFs), Hardened DSP slices, and Block RAMs)
 - Unlike GCC, FPGA HLS compilers like Vitis HLS is “Target-Aware”
 - Contains internal databases of the exact routing delays, multiplexer structures, and resource counts of specific FPGA
 - Maps data to BRAM for high memory bandwidth
- 

Mathematical Modeling & Scheduling for FPGAs

The SDC Framework ($O(n)$ Complexity)

- Assigns a single continuous variable (s) to represent the start clock cycle of each operation.

Modeling Data Dependencies

- Operations are mathematically separated by constant delays (d).
- $s_j - s_i \leq d$ (Operation j must wait for i to finish)

Modeling FPGA Resource Limits

- The scheduler cannot exceed the exact physical limits of the target FPGA fabric (DSPs, LUTs, BRAMs).
- $Gs \leq p$ (*Scheduled Resources < Available Chip Resources*)

The Total Unimodularity Guarantee

- **Objective:** $c^T s$ (Minimize Latency)
- SDC constraint matrix guarantees that the standard LP solver will natively return fast, **whole-integer clock cycles** without expensive algorithms.

The HLS Algorithm and Compilation Flow

Front-End Compilation: Modern tools parse C/C++ code into an Intermediate Representation (IR) utilizing compiler infrastructures such as LLVM.

- Algorithms like Static Single Assignment (SSA) are strictly employed for constant propagation and dead code elimination.

Optimizing Soft Constraints: The scheduler algorithm incorporates "soft" constraints (e.g., pipeline stage preferences) by introducing a violation variable (v_j) and a penalty term ($\phi(v_j)$) into the objective function.

- **Modified Objective:** Minimize $c^T s + \sum \phi(v_j)$.
- **Subject to:** $H_j s - v_j$ and $-v_j \leq 0$.

Datapath Generation: Ultimately, scheduling, allocation, and binding work concurrently to generate RTL from a control-and-datapath flow graph (CDFG).

Runtime, Complexity, and Current Advancements

Scalable Compilation (The Power of SDC)

- The mathematical "unimodularity" guarantees fast, whole-integer clock cycles.
- Avoids slow, exponential problem-solving, allowing HLS to compile even massive, modern FPGA designs in minutes.

The Compiler Trade-off

- Tools must balance squeezing out the absolute maximum hardware efficiency (lowest LUT/DSP count) against keeping compile times fast and the output RTL readable.

Runtime, Complexity, and Current Advancements

Modern FPGA Advancements

- **Task-Parallelism:** Moving beyond simple `for` loops to design complex, concurrent streaming data paths directly in C++.
- **Rapid Debugging:** Verifying logic using instantaneous C++ software simulation instead of agonizingly slow RTL waveform simulators.
- **AI/ML Integration:** Tools now use Machine Learning to predict physical routing congestion on the FPGA silicon *before* standard synthesis even begins.

Unrolling Effects on the hardware area of an FPGA

Multiplying Hardware Components: Degree of parallelism, DOP_k

The Physical Area Constraint (A): optimized area must strictly remain less than the total available device area: $A_{lu} < A$

$$A_{lu} = \sum_{k=1}^K (DOP_k \times a_k) + A_{control}$$

FPGA Reality Check: The DSP Hard Limit

IDEAL SCENARIO
(COMPLETE UNROLLING)

768
DSPs REQUIRED



For simulated 32-bit integer 3x3 Filter, max efficiency.

REALITY:
TARGET DEVICE CAPACITY

740
DSPs AVAILABLE
(xc7a200t)



Physical silicon grid constraint; resource deficit.

CONSEQUENCE: MANDATORY
SHIFT TO PARTIAL UNROLLING

FIT ACHIEVED



Because "Ideal" exceeds capacity, developers must settle for partial unrolling to fit implementation.

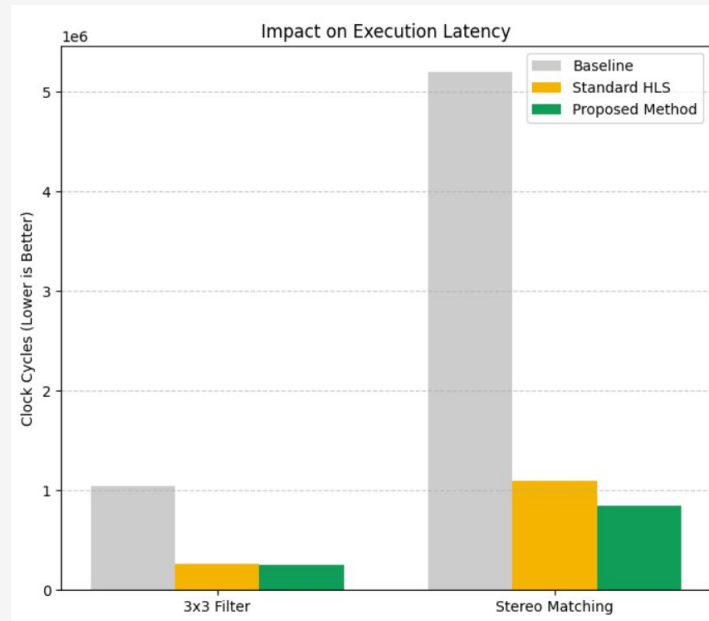
Simulated Results: Code Optimization in Practice

The Benchmarks

- Evaluated across two real-time image processing algorithms (3 X 3 Filter, Stereo Matching).

Reduced Latency (Faster Execution)

- Simplifying the C/C++ loop hierarchies exposed deeper instruction-level pipelining opportunities for the HLS scheduler.



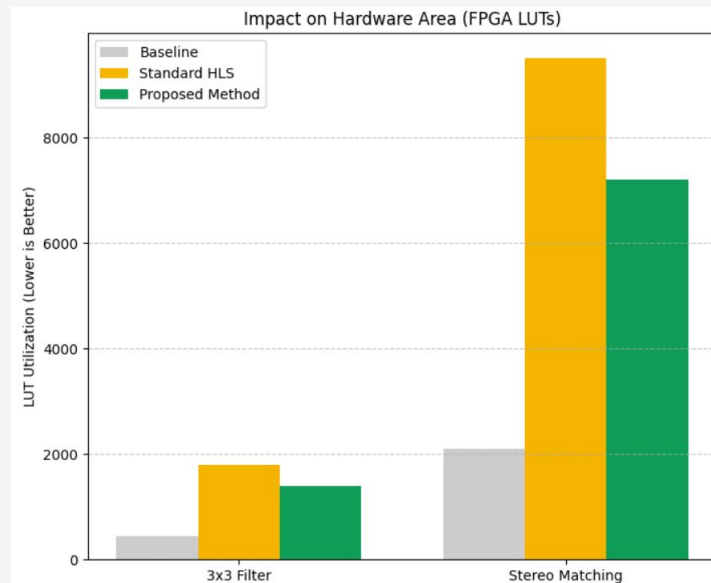
Simulated Results: Code Optimization in Practice

Reduced Hardware Area (Fewer Resources)

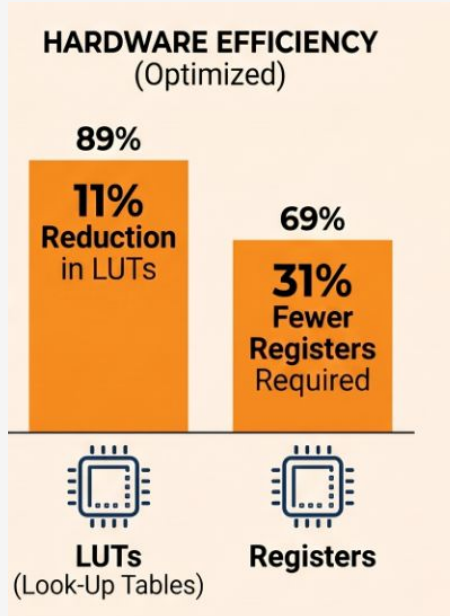
- Cleaner mathematical representation of the code required significantly less complex control logic (fewer LUTs/Registers) during synthesis.

Proposed Method

- Combining intelligent source-code restructuring *with* standard HLS pragmas yields much higher efficiency than relying on the compiler alone.



Realization of Application Examples



Case Study 1: MIMO Sphere Decoder: A multi-input multi-output wireless system application.

- *Architecture:* HLS achieves a highly parallel streaming dataflow architecture.
- *Results:* Yielded an 11% reduction in Look-Up Tables (LUTs) and a 31% reduction in registers compared to hand-coded RTL, in roughly the same or less development time.

Conclusion

Continual algorithmic advancements have made HLS fiercely competitive with manual RTL. Modern tools provide excellent quality of results (QoR) and resource utilization, genuinely saving time and effort compared to writing VHDL or Verilog manually.

[8]

...

Questions?

References:

1. <https://www.vlsi-expert.com/2021/05/high-level-synthesis-intro.html> [Accessed: Jun. 22, 2026].
2. Y. Chi, L. Guo, J. Lau, Y. Choi, J. Wang, and J. Cong, "Extending High-Level Synthesis for Task-Parallel Programs," in 2021 IEEE 29th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM), 2021, pp. 204-213, doi: 10.1109/fccm51124.2021.00032
3. C. Li, Y. Bi, Y. Benezeth, D. Ginjac, and F. Yang, "High-level synthesis for FPGAs: code optimization strategies for real-time image processing," Journal of Real-Time Image Processing, vol. 14, pp. 701-712, 2018, doi: 10.1007/s11554-017-0722-3.
4. Z. Kou et al., "High-Level Synthesis Design of Scalable Ultrafast Ultrasound Beamformer With Single FPGA," IEEE Transactions on Biomedical Circuits and Systems, vol. 17, pp. 446-457, 2023, doi: 10.1109/TBCAS.2023.3267614.
5. J. Cong, B. Liu, S. Neuendorffer, J. Noguera, K. Vissers, and Z. Zhang, "High-Level Synthesis for FPGAs: From Prototyping to Deployment," IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 30, no. 4, pp. 473-491, Apr. 2011, doi: 0.1109/TCAD.2011.2110592
6. .
7. S. Lahti and T. D. Hämäläinen, "High-Level Synthesis for FPGAs—A Hardware Engineer's Perspective," IEEE Access, vol. 13, pp. 28574-28593, 2025, doi: 10.1109/ACCESS.2025.3540320
8. LLM generated image